

Aflevering P3

Mikkel Moth Billing

Denne aflevering indeholder to opgaver, der kan løses uafhængigt.

Første opgave er en Sangaku-opgave, hvor du gentager en opgaveløsning som du har været igennem en del gange til TØ og i de to tidligere afleveringer.

Den anden opgave er en opgave inspireret af de opgaver du lavede i kapitel 13 i kurset Calculus i sidste semester.

Spørgsmål 1

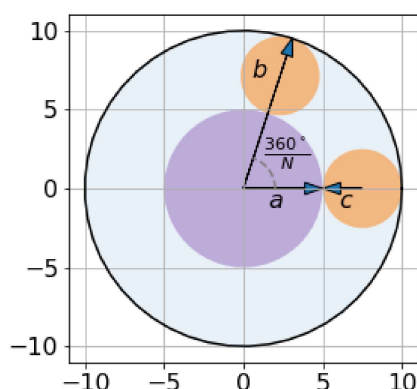
Gengiv den feedback du fik i forbindelse med Aflevering P2 og svar på evt. spørgsmål instruktoren har stillet.

Fra Aflevering P2 fik jeg at vide at jeg kunne bruge rotationsmatricen til at rotere cirklerne rundt om firkanten.

Sangaku-opgaven

Vi skal kigge på Sangaku-tegninger, hvor området mellem to koncentriske skiver (= skiver med samme centrum) er fyldt af N cirkler med samme radius, der netop rører hinanden. De algebraiske udtryk for forholdene mellem cirklernes radier blev løst og dokumenteret som Sangaku-tegninger i det sene 19. århundrede (1879 for $N = 8$ og 1891 for $N = 5$).

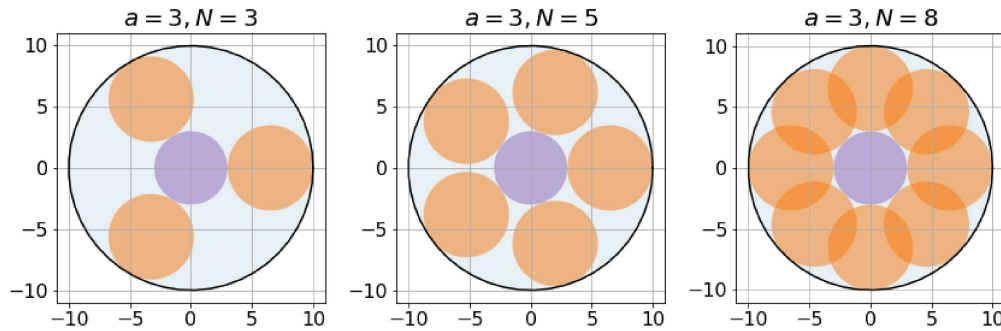
Lad de to koncentriske cirkler have radius henholdsvis a og b . Lad radius af de N cirkler være c . Hvis man kender værdierne for a og b følger værdien af c . Her er en figur til at hjælpe dig med at forstå geometrien i problemet.



I figuren har a og b endnu ikke de rigtige værdier i forhold til hinanden, hvilket betyder at de to afbildede cirkler med radius c ikke rører hinanden som de skal i de endelige Sangaku-tegninger. Det vender vi tilbage til senere, og starter med at antage at de N cirkler med radius c er jævnt fordelt i området mellem de koncentriske cirkler.

Spørgsmål 2

Udled et udtryk for c når a og b er kendte. Lav en samlet figur hvor du plotter for $a = 3, b = 10$ og N værende 3, 5 og 8.



```
In [1]: import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import fmin

plt.rc("xtick", labels=10)
plt.rc("ytick", labels=10)
plt.rc("axes", titlesize=16)
plt.rc("font", size=16)
plt.rc("axes", grid='on')
```

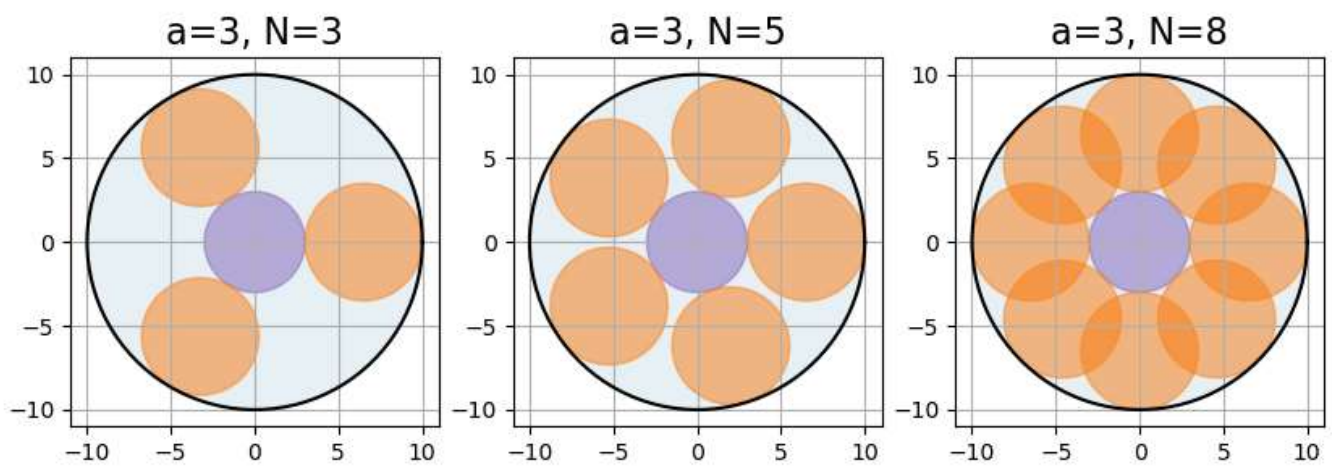
```
In [2]: cf = lambda a,b: (b-a)/2

a_l = [3,3,3]
b_l = [10,10,10]
N_l = [3,5,8]

def fig(a_l,b_l,N_l, a_color="purple", c_color="orange"):
    fig, axes = plt.subplots(1,3, figsize=(10,8))
    axes.flatten()
    t = np.linspace(0,2*np.pi,100)
    R = lambda v: np.array([[np.cos(np.pi/180*v), -np.sin(np.pi/180*v)],
                             [np.sin(np.pi/180*v), np.cos(np.pi/180*v)]])
    for ax, a,b,N in zip(axes,a_l,b_l,N_l):
        c = cf(a,b)
        ax.fill(a*np.cos(t), a*np.sin(t), color=f"tab:{a_color}", alpha=0.5)
        ax.fill(b*np.cos(t), b*np.sin(t), color="tab:blue", alpha=0.1)
        ax.plot(b*np.cos(t), b*np.sin(t), "k")
        cm = np.array([a+c, 0])
        for _ in range(N):
            rt = 360/N
            ax.fill(c*np.cos(t)+cm[0], c*np.sin(t)+cm[1], color=f"tab:{c_color}", alpha=0.5)
            cm = np.dot(R(rt),cm)

        ax.set_title(f"a={a:.0f}, N={N}")
        ax.set_aspect('equal')
        ax.xaxis.set_major_locator(plt.MultipleLocator(5))
        ax.yaxis.set_major_locator(plt.MultipleLocator(5))

fig(a_l, b_l, N_l)
```



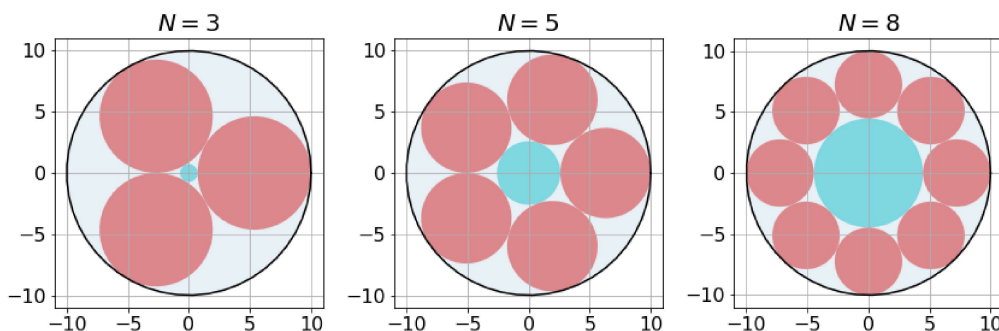
Spørgsmål 3

Opskriv et udtryk for afstanden mellem to nabo-cirkler (altså af dem med radius c). Det er afstanden mellem de to orange cirkler på den første figur i opgaven). Skriv en python-funktion, der når a , b og N er kendte beregner først c og dernæst returnerer:

$$\text{Loss}(a, b, N) = |d_{\text{faktisk}}(a, b, N) - d_{\text{ønsket}}(a, b)|^2$$

hvor d_{faktisk} er den faktiske afstand mellem centrene af to nabo-cirkler mens $d_{\text{ønsket}}$ er den ønskede afstand mellem centrene (= den, der skal til for at de netop rører hinanden). Du kan nu til enhver tid finde den rigtige værdi af a når du har valgt værdier for b og N . Det gør du ved at minimere Loss med `scipy.optimize.fmin`.

Lav en samlet figur hvor du plotter for N værende 3, 5 og 8. Lad $b = 10$.



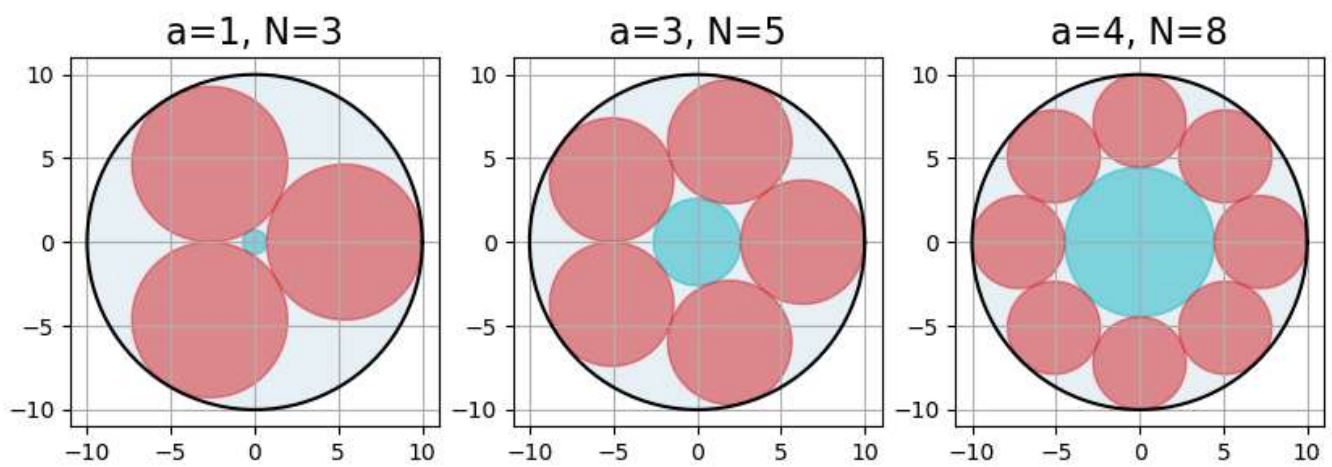
$$a^2 = b^2 + c^2 - 2bc \cdot \cos(A)$$

$$\text{Afstand} = \sqrt{2(a+c)^2 - 2(a+c)^2 \cdot \cos(360/N)} = (a+c) \cdot \sqrt{-2 \cdot \cos(360/N) + 2}$$

```
In [3]: d_f = lambda a,b,N: (a+cf(a,b))*np.sqrt(-2*np.cos(360/N*np.pi/180)+2)
Loss = lambda a,b,N: abs(d_f(a,b,N)-2*cf(a,b))**2
al = []
for b,N in zip(b_l, N_l):
    al.append(fmin(lambda a: Loss(a, b,N),0,disp=False)[0])

fig(al,b_l,N_l, "cyan", "red")
print(al)
```

[0.7179375000000008, 2.5961875000000028, 4.464625000000005]



Calculus-opgaven

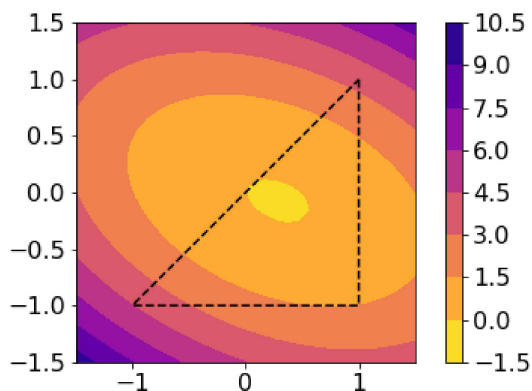
En funktion,

$$f: f(x, y) = x^2 + 2y^2 - \frac{1}{2}x + xy$$

er defineret området: $\Omega = \{(x, y) : -1 \leq x \leq 1, -1 \leq y \leq x\}$.

Spørgsmål 4

Lav et konturplot af funktionen og afsæt randen af Ω med en stiplede linie:



```
In [4]: f = lambda x,y: x**2+2*y**2-1/2*x+x*y
xs = np.linspace(-1.5, 1.5, 100)
ys = np.linspace(-1.5, 1.5, 100)
X, Y = np.meshgrid(xs,ys)

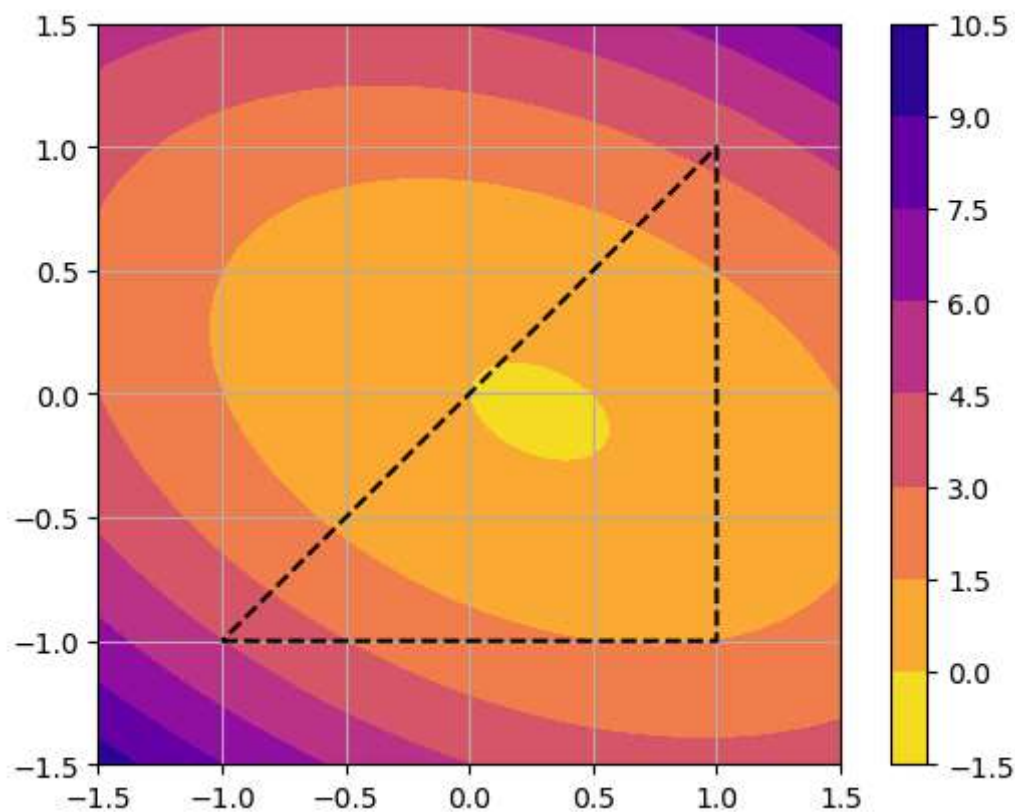
Z = f(X, Y)
levels = np.arange(-1.5, 12, 1.5)

fig, ax = plt.subplots()

color_map = ax.contourf(X, Y, Z, levels=levels, cmap='plasma_r')
fig.colorbar(color_map, ax=ax)

ax.plot([-1,1,1,-1], [-1,-1,1,-1], "k--")

ax.set_aspect('equal')
```

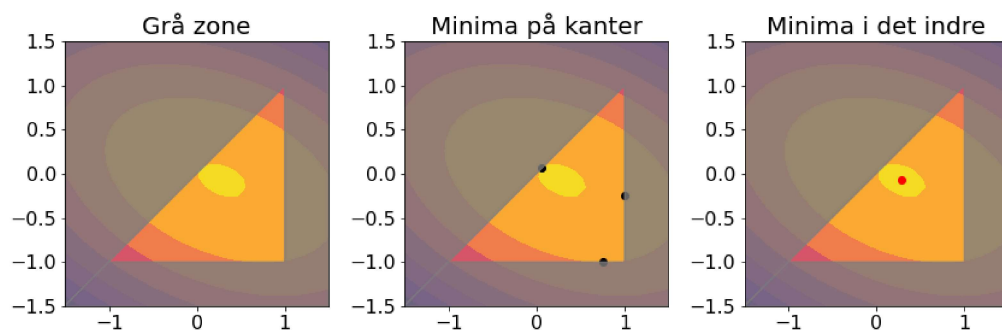


Spørgsmål 5

Tegn nu i en samlet figur konturplottet tre gange idet du skærmer området uden for Ω af med en grå farve, der er svagt gennemsigtig.

I det midterste plot skal du nu foretage den sædvanlige analyse af hvor der er minima på randen. Det gør du ved for hvert af de tre liniestykker at opskrive en NumPy-funktion, af kun én variabel og minimere den med `scipy.optimize.fmin`. Du skal ikke lave calculus selv, men opskrive disse én-dimensionelle funktioner ved at genbruge den 2-dimensionelle NumPy-funktion du indførte i spørgsmål 4 for at lave konturplottet.

Til sidst skal du fortage en minimering af f i begge dimensioner x og y og dermed finde minimummet inde i Ω . Afbild dette i det sidste plot.



```
In [5]: fig, axes = plt.subplots(1,3, figsize=(10,4))

for ax in axes:
    color_map = ax.contourf(X, Y, Z, levels=levels, cmap='plasma_r')
    ax.fill([-1.5, -1, 1, 1, -1.5, -1.5, 1.5, 1.5],
           [-1.5, -1, -1, 1, -1.5, 1.5, 1.5, -1.5],
```

```

        color="gray", alpha=0.8)

    ax.grid(False)
    ax.set_aspect('equal')

    f1 = [lambda x: f(x,-1),
          lambda y: f(1,y),
          lambda x: f(x,x)]

    x1 = fmin(f1[0], 0, disp=False)[0]
    y1 = fmin(f1[1], 0, disp=False)[0]
    m = fmin(f1[2], 0, disp=False)[0]

    axes[1].scatter([x1,1,m], [-1,y1,m], color="black")

    P0 = (0,0)
    P = fmin(lambda P: f(P[0],P[1]), P0, disp=False)
    axes[2].scatter(P[0],P[1], color="red")
    plt.show()

```

